

Задание 2. Архитектурное решение для системы бронирования

1. Постановка задачи

Условие

Имеется система бронирования столиков в ресторанах. Состоит она из двух микросервисов:

1. Сервис бронирований - принимает запросы от пользователей, создаёт бронь.
2. Сервис ресторанов - хранит информацию о ресторанах и свободных столиках.

Система устроена так:

Чтобы забронировать столик, мобильное приложение делает запрос на сервис бронирования. При каждом бронировании сервис бронирований синхронно вызывает сервис ресторанов по REST, чтобы проверить доступность столика и занять его. Если вызов завершается таймаутом или ошибкой, начальный запрос пользователя тоже завершается ошибкой.

Эти микросервисы ни за что другое больше не отвечают.

Обычная нагрузка на сервис бронирования - 100 запросов в секунду. Однако иногда в ресторанах этой компании обедают знаменитости и непременно пишут об этом в своих социальных сетях.

Сразу после любого такого поста, нагрузка возрастает ровно до 50 000 запросов на бронирование в секунду. И держится на таких значениях в течение 10-15 минут. После этого падает до нормальных 100 запросов в секунду.

Естественно, при пиковой нагрузке вся система моментально падает.

Интересно, что сервис бронирования и сервис ресторанов запущены в разных ДатаЦентрах (ДЦ). Микросервис бронирования запущен в большом кол-ве экземпляров в ДЦ1, а микросервис ресторанов работает в единственном экземпляре в ДЦ2. Причём сервис ресторанов делался отдельной компанией, которая отказывается поднимать дополнительные экземпляры этого микросервиса и так же не даёт исходный код, чтобы мы сделали это сами (это жесткое ограничение на задание - нельзя запускать новые экземпляры сервиса ресторанов. Есть только 1 начальный. На микросервис бронирования таких ограничений НЕТ. Их можно поднимать в любом кол-ве экземпляров).

Мобильное приложение знает адреса всех экземпляров сервиса бронирования и само решает, на какой экземпляр отправить запрос. Адреса защиты в коде приложения в одном и том же порядке у всех пользователей. Приложение действует просто - берёт первый попавшийся ip адрес и делает запрос на него. Если сервис недоступен, идёт на второй. И так далее по списку. Если все сервисы лежат, оно начинает заново проходиться по списку сверху вниз. (То есть при пиковой нагрузке все запросы идёт вначале только на первый экземпляр сервиса, потом только на второй и так далее).

Каждый раз, когда случается пиковая нагрузка, все сервисы бронирования падают под нагрузкой. Также известно, что если бы сервисы бронирования выстояли эту нагрузку, то упал бы сервис ресторанов - он не способен одновременно обрабатывать такое кол-во запросов.

Отдел аналитиков выяснил, что невозможно предсказать в какие конкретно моменты времени и дни недели знаменитости будут постить - это происходит абсолютно случайно. Однако они заметили, что если бы нагрузка на микросервис ресторанов была бы примерно равномерной в течение дня, то он бы спокойно обрабатывал все входящие запросы, при этом потреблял бы всего 20% от своих максимальных мощностей.

Разрешается, чтобы бронирование столика происходило не моментально - пользователь оставляет заявку в приложении и она должна быть обработана в течение 24 часов. Также, при необходимости, в ДЦ1 можно поднимать дополнительные экземпляры сервиса бронирования (в неограниченном кол-ве). Также разрешается поднимать какие либо другие вспомогательные сервисы, не связанные напрямую с данной системой.

Также важно отметить, что рестораны по размеру бесконечные (свободные столики всегда есть в достаточном количестве). В них вполне можно разместить всех пользователей, которые делают запросы на бронирование в течении пиковой нагрузки. В связи с этим бизнес запрещает отвечать отказами на бронирование в пиковые нагрузки.

Считаем, что пропускная способность сети на вход и выход ДЦ1 и ДЦ2, а также между ДЦ1 и ДЦ2 спокойно выдержит все запросы пользователей.

Для полного балла опишите 2 центральные проблемы, присутствующие в описанной архитектуре системы. И предложите любой из способов устранения для каждой из них.

Краткий анализ исходной системы

Система падает не из-за одной ошибки, а из-за двух архитектурных проблем: синхронной зависимости от единственного сервиса ресторанов и неправильной балансировки нагрузки на сервис бронирований. Решение: принимать заявки через единый вход, быстро сохранять их в надежное хранилище, складывать в очередь и постепенно обрабатывать с ограничением скорости вызовов к сервису ресторанов.

Расчет нагрузки и SLA

Пиковая нагрузка за 15 минут дает $50\,000 \text{ запросов/сек} \times 900 \text{ секунд} = 45\,000\,000$ заявок. Если обработать этот объем за 24 часа, средняя скорость обработки должна быть примерно 521 заявка/сек. Это подтверждает, что задачу нужно решать сглаживанием нагрузки через очередь, а не синхронной обработкой всего пика в момент запроса.

Проблема 1. Синхронная зависимость от сервиса ресторанов

Сервис бронирований при каждом запросе синхронно вызывает сервис ресторанов по REST. Поэтому пользовательский запрос считается успешным только тогда, когда сервис ресторанов успел ответить.

В чем проблема

- Пиковая нагрузка 50 000 запросов в секунду напрямую превращается в 50 000 синхронных вызовов к сервису ресторанов.
- Сервис ресторанов существует в одном экземпляре в другом дата-центре, и масштабировать его по условию нельзя.
- Если сервис ресторанов отвечает медленно, падает или уходит в таймаут, запрос пользователя тоже завершается ошибкой.
- Масштабирование сервиса бронирований само по себе не решает проблему: все экземпляры все равно будут давить на один и тот же сервис ресторанов.
- По условию пользователю можно не подтверждать бронь мгновенно, а свободных столиков всегда достаточно. Значит, нельзя отказывать из-за технического пика; нужно принять заявку и подтвердить ее позже.

Решение 1. Асинхронная обработка через очередь

Нужно разделить прием заявки от фактического обращения к сервису ресторанов.

1. Booking Service принимает запрос, выполняет только легкую валидацию и короткой транзакцией сохраняет заявку в своей базе данных.
2. Заявка получает статус NEW и помещается в персистентную очередь, например RabbitMQ, Kafka, SQS, либо в outbox-таблицу с последующей публикацией в очередь.
3. Пользователь сразу получает ответ 202 Accepted и request_id. Это подтверждение принятия заявки, а не финальное подтверждение брони.
4. Фоновые воркеры постепенно читают очередь и вызывают Restaurant Service с контролируемой скоростью.
5. Перед вызовом Restaurant Service используется rate limiter, чтобы единственный экземпляр получал равномерную нагрузку и не перегружался.
6. При таймаутах и временных ошибках заявка не теряется: выполняются повторные попытки с backoff. После исчерпания попыток заявка попадает в MANUAL_REVIEW или DLQ для ручной обработки.
7. Так как очередь обычно дает at-least-once обработку, обработка заявки должна быть идемпотентной: повторная попытка не должна создавать вторую бронь.

Статусы заявки

Статус	Смысл
NEW	Заявка принята и ожидает обработки.
PROCESSING	Воркер обрабатывает заявку.
CONFIRMED	Бронь успешно подтверждена через сервис ресторанов.
MANUAL_REVIEW	Автоматическая обработка не завершилась, требуется ручная проверка.

Повторы лучше хранить не как отдельный бизнес-статус, а как технические поля retry_count, next_retry_at и last_error.

Почему это устраняет проблему

- Пик пользовательских запросов поглощается очередью, а не передается напрямую в Restaurant Service.
- Сервис ресторанов получает ровную управляемую нагрузку, которую он способен обработать.
- Заявки не теряются при сбоях, потому что сначала сохраняются в надежное хранилище и только потом обрабатываются асинхронно.
- Бизнес-требование соблюдается: пользователям не отказывают в пиковую нагрузку, а возраст самой старой необработанной заявки должен оставаться меньше 24 часов.

Проблема 2. Клиентская балансировка по зашитому списку IP

Мобильное приложение хранит адреса всех экземпляров сервиса бронирований и у всех пользователей проходит их в одном и том же порядке. При пике весь поток сначала идет на первый экземпляр, после его падения — на второй, затем на третий и так далее.

В чем проблема

- Нагрузка распределяется не равномерно, а последовательно обрушивается на отдельные экземпляры.
- Большое количество экземпляров Booking Service не помогает, потому что приложение не использует их одновременно.
- Клиент знает внутреннюю топологию системы, из-за чего изменения инфраструктуры требуют обновления приложения.
- Отказ одного экземпляра вызывает лавинообразный перенос нагрузки на следующий экземпляр.

Решение 2. Единый вход через Load Balancer или API Gateway

Мобильное приложение не должно выбирать конкретный экземпляр сервиса. В приложении должен остаться один стабильный адрес API, например `api.booking.company.com`.

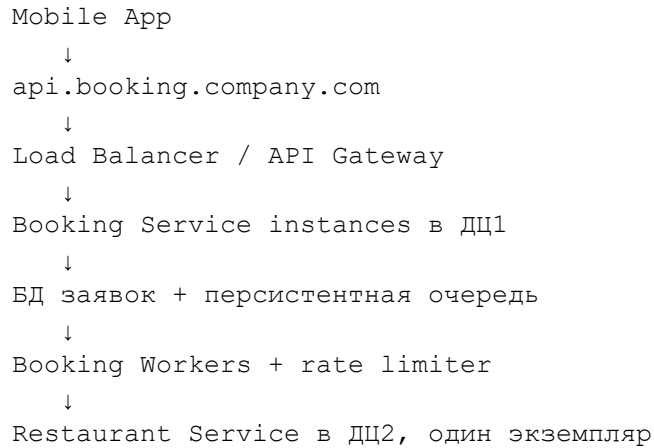
1. Все запросы пользователей приходят на Load Balancer или API Gateway в ДЦ1.
2. Балансировщик равномерно распределяет запросы между доступными экземплярами Booking Service.
3. Health checks исключают из балансировки неработающие экземпляры.
4. Экземпляры Booking Service можно добавлять и удалять без изменения мобильного приложения.
5. Autoscaling в ДЦ1 позволяет быстро увеличить количество экземпляров Booking Service во время пиков.
6. Для старых версий мобильного приложения зашитые IP можно перевести на reverse проху или VIP-адреса, чтобы даже старые клиенты попадали в балансируемый вход.

Почему это устраняет проблему

- Пиковый поток не попадает в один экземпляр, а распределяется между всеми рабочими экземплярами.
- Падение одного экземпляра не ломает весь прием заявок.
- Инфраструктура становится управляемой на стороне сервера, а не внутри мобильного приложения.

- Booking Service можно масштабировать горизонтально, а основную защиту от перегрузки Restaurant Service выполняет очередь.

Целевая схема обработки



Дополнительные элементы надежности

- Idempotency key или client_request_id: повторный запрос от мобильного приложения не создает вторую бронь, а возвращает статус уже созданной заявки. Это важно, потому что приложение может повторять запросы при таймаутах.
- Отдельный endpoint проверки статуса: пользователь получает request_id и может запросить состояние заявки через GET /booking-requests/{id}.
- Уведомление о результате: после подтверждения заявки система может отправить push, email или SMS.
- Мониторинг очереди: нужно отслеживать размер очереди, возраст самой старой заявки, число ошибок, скорость обработки и время обработки. Главный SLA-показатель — самая старая необработанная заявка должна быть младше 24 часов.

Ограничения решения

Масштабирование только сервиса бронирований не решает задачу, так как единственный сервис ресторанов остается узким местом. Отказы пользователям также не подходят, поскольку по условию все заявки должны быть приняты и обработаны. Поэтому основное решение должно сочетать единый вход через балансировщик, быстрое сохранение заявок, персистентную очередь, идемпотентную обработку и контролируемую скорость вызовов к Restaurant Service.

Результат работы

Итог: мобильное приложение отправляет запрос на единый API; балансировщик распределяет поток; Booking Service сохраняет заявку и возвращает request_id; бронирование выполняется позже через очередь и воркеры с rate limiter к Restaurant Service. Это устраняет две причины падения: неравномерный входящий трафик и перегрузку единственного сервиса ресторанов.